

My Contribution and AI Assistance

1. My Contribution

I worked on the complete clinical machine learning threshold optimization project using Google Colab. My main contribution was implementing the machine learning workflow, running the experiments on the actual dataset, checking the outputs, comparing different threshold strategies, and making the final threshold decision based on validation results.

I also verified that the test set was not used during model development and was reserved only for the final evaluation.

2. Project Implementation

2.1 Dataset Loading and Initial Analysis

I started by loading the clinical dataset into Google Colab using Pandas.

The dataset contained:

- 280,985 rows
- 39 columns
- Clinical, demographic, and disease-related variables
- Target variable: label

The target contained two classes:

- Normal
- Abnormal

I converted the target into binary format:

- Normal = 0
- Abnormal = 1

I then checked:

- Dataset shape
- Column names
- Data types
- Missing values

- Class distribution
- Duplicate records
- Possible invalid values

This helped me understand the structure and quality of the dataset before model training.

2.2 Data Quality and Duplicate Analysis

I checked the dataset for duplicate records and found repeated feature rows.

There were:

- 2,284 exact duplicate feature rows
- Approximately 0.81% of the dataset

These exact duplicate feature rows were removed to avoid giving repeated observations unnecessary influence during model training.

The `composite_key` column was not used as a duplicate criterion because it was treated as an identifier rather than a clinical feature.

After duplicate removal, the dataset contained approximately:

- 278,701 observations

The class distribution remained reasonably consistent, with approximately:

- 62.45% Abnormal
- 37.55% Normal

2.3 Data Leakage Investigation

Before training the model, I specifically investigated possible data leakage.

Some columns contained information that could directly represent the target or its underlying disease categories. These included:

- **sublabel**
- **disease_flags**
- **source_dataset**
- **composite_key**

These columns were removed from the predictive feature set.

The target column label was also kept completely separate from the input features.

Clinical variables such as diabetes, hypertension, heart disease, glucose, BMI, blood pressure, and other medical measurements were retained because they can act as legitimate predictors.

This step was important because a model with leaked target information could produce unrealistically high performance.

2.4 Feature Preparation

After leakage and unnecessary columns were removed, the remaining predictive features were divided into:

- Numerical features
- Categorical features

There were:

- 20 numerical features
- 14 categorical features

The numerical features included clinical measurements such as:

- Age
- BMI
- Glucose
- HbA1c
- Cholesterol
- Blood pressure
- Heart rate
- Triglycerides
- HDL
- LDL

Categorical features included variables such as:

- Gender
- Smoking
- Diabetes status

- Hypertension status
- Physical activity
- Family history
- Education level
- Employment status

3. Preprocessing Pipeline

I created a preprocessing pipeline so that the same transformation process could be consistently applied to the training, validation, and test data.

3.1 Numerical Features

For numerical features, I used:

StandardScaler

This standardized numerical variables so that features with larger numerical ranges would not dominate the Logistic Regression model.

3.2 Categorical Features

For categorical features, I used:

OneHotEncoder

I used:

`handle_unknown="ignore"`

This prevents errors if a category appears in validation or test data that was not observed during training.

3.3 ColumnTransformer

I combined numerical and categorical preprocessing using ColumnTransformer.

This created one complete preprocessing pipeline instead of manually transforming different datasets separately.

4. Train, Validation, and Test Split

I divided the dataset into three separate parts using stratified splitting:

- 60% Training
- 20% Validation
- 20% Test

The final sizes were approximately:

- Training: 167,220
- Validation: 55,740
- Test: 55,741

Stratification was used so that the Normal and Abnormal class proportions remained similar across all three datasets.

The purpose of the three-way split was:

Training Set

Used for:

- Model training
- Learning preprocessing parameters
- Calibration training

Validation Set

Used for:

- Model evaluation during development
- Calibration comparison
- Threshold sweeping
- Clinical threshold optimization
- Bootstrap validation
- Threshold stability analysis
- Final threshold selection

Test Set

Reserved completely for:

- Final holdout evaluation only

The test set was not used for threshold selection, calibration, optimization, or tuning.

5. Baseline Model

I selected Logistic Regression as the baseline classifier.

The model was appropriate because:

- It is a strong and interpretable baseline for binary classification.
- It produces probability estimates.
- It works efficiently with a large dataset.
- It works well with standardized numerical features and one-hot encoded categorical features.

The baseline classification threshold was:

0.50

At this threshold, probabilities of 0.50 or higher were classified as Abnormal.

The main clinical objective was to prioritize Recall because missing an Abnormal case can be more important than generating an additional false positive in a screening-style setting.

The minimum target Recall was defined as:

95%

6. Baseline Model Evaluation

I evaluated the baseline classifier using multiple classification metrics instead of relying only on Accuracy.

The metrics included:

- Accuracy
- Precision
- Recall
- F1-Score
- Specificity
- ROC-AUC
- PR-AUC

I also calculated the confusion matrix containing:

- True Negatives
- False Positives
- False Negatives
- True Positives

This provided a more complete understanding of the model's clinical behavior.

7. Probability Calibration

Because threshold optimization depends on predicted probabilities, I evaluated whether the model probabilities were well calibrated.

I compared three approaches:

1. Uncalibrated Logistic Regression
2. Sigmoid Calibration
3. Isotonic Calibration

I evaluated calibration using:

- Brier Score
- Log Loss

The results were:

Method	Brier Score	Log Loss
Uncalibrated	0.0929	0.2830
Sigmoid	0.0929	0.2830
Isotonic	0.0870	0.2608

Isotonic calibration produced the lowest Brier Score and Log Loss.

Therefore, I selected the Isotonic calibrated model for the threshold optimization stage.

The calibration process was performed without using the final test set.

8. Threshold Sweeping

After obtaining calibrated probabilities, I systematically tested different classification thresholds.

The threshold range was:

0.00 to 1.00

with a step size of:

0.01

This produced 101 threshold values.

For every threshold, I converted predicted probabilities into class predictions and calculated:

- True Positives
- True Negatives
- False Positives
- False Negatives
- Precision
- Recall
- F1-Score
- Specificity
- False Positive Rate
- False Negative Rate

This created a complete threshold performance DataFrame.

The purpose was to understand how changing the decision threshold changes clinical classification behavior.

9. Precision, Recall and F1 Trade-off

I plotted the main classification metrics against the threshold.

The analysis showed the expected trade-off:

Lower Threshold

A lower threshold makes it easier for the model to classify a case as Abnormal.

This generally results in:

- Higher Recall
- Fewer False Negatives
- More False Positives
- Lower Specificity

Higher Threshold

A higher threshold makes the model more conservative when predicting Abnormal.

This generally results in:

- Higher Precision
- Higher Specificity
- Fewer False Positives
- More False Negatives
- Lower Recall

This demonstrated why a threshold should not automatically be fixed at 0.50 for every clinical problem.

10. Maximum F1 Threshold

I identified the threshold that produced the highest F1-Score.

The maximum-F1 threshold was:

0.49

Its validation performance was approximately:

- Precision: 0.8612
- Recall: 0.9518
- F1-Score: 0.9042
- Specificity: 0.7448

The F1-Score was only slightly higher than the result at 0.50.

Therefore, maximum F1 alone was not used as the final clinical decision rule.

11. Precision-Recall Balance Point

I also identified the threshold where Precision and Recall were closest to each other.

The closest balance occurred around:

0.66

with approximately:

- Precision: 0.8906
- Recall: 0.8868

This showed a more balanced Precision-Recall operating point, but Recall was below the required 95% clinical target.

Therefore, this threshold was not selected for the final screening objective.

12. Clinical Threshold Optimization

Instead of selecting a threshold only from general metrics, I implemented several clinical optimization strategies.

12.1 Recall-Constrained Optimization

The minimum required Recall was:

95%

I searched for thresholds that satisfied this requirement.

The selected threshold was:

0.50

Validation performance:

- Precision: 0.8622
- Recall: 0.9502
- F1-Score: 0.9041
- Specificity: 0.7474

Threshold 0.50 was preferred over 0.49 because it still satisfied the Recall requirement while providing slightly better Precision and Specificity.

12.2 Precision-Constrained Optimization

I also tested a Precision-focused scenario.

The minimum Precision requirement was:

90%

The selected threshold was:

0.70

Validation performance:

- Precision: 0.9063
- Recall: 0.8429
- F1-Score: 0.8735
- Specificity: 0.8551

This demonstrated that increasing Precision requires accepting lower Recall.

13. Cost-Sensitive Threshold Optimization

I then implemented a cost-sensitive approach.

For a clinical screening scenario, I considered False Negatives more expensive than False Positives.

The selected costs were:

- False Negative cost = 10

- False Positive cost = 1

The threshold producing the lowest total classification cost was:

0.08

Performance:

- Precision: 0.7734
- Recall: 0.9951
- F1-Score: 0.8704
- Specificity: 0.5152
- False Negatives: 171
- False Positives: 10,147
- Total Cost: 11,857

This showed that strongly prioritizing the avoidance of False Negatives can require accepting a large number of False Positives.

14. Multiple Clinical Cost Scenarios

I tested different cost assumptions to understand how clinical priorities affect the optimal threshold.

Scenario	FN Cost	FP Cost	Threshold
FN-Sensitive	10	1	0.08
Balanced	5	5	0.50
FP-Sensitive	1	5	0.86
Highly FN-Sensitive	20	1	0.04

This demonstrated that there is no universally optimal threshold.

The optimal threshold depends on:

- Clinical risk
- Cost of missed cases
- Cost of false alarms
- Available healthcare resources
- Intended use of the model

15. Comparison of Threshold Strategies

I compared all major threshold selection strategies.

Objective	Threshold	Precision	Recall	F1-Score	Specificity
Default	0.50	0.8622	0.9502	0.9041	0.7474
Maximum F1	0.49	0.8612	0.9518	0.9042	0.7448
Recall-Constrained	0.50	0.8622	0.9502	0.9041	0.7474
Precision-Constrained	0.70	0.9063	0.8429	0.8735	0.8551
Cost-Optimal	0.08	0.7734	0.9951	0.8704	0.5152

Based on the stated clinical objective, threshold 0.50 was selected.

16. Bootstrap Statistical Validation

To evaluate the reliability of the validation metrics, I implemented bootstrap confidence intervals.

I used:

- 1,000 bootstrap samples
- 95% percentile confidence intervals
- Validation data
- Threshold = 0.50

The results were:

Metric	Estimate	95% CI
Precision	0.8622	0.8588–0.8655
Recall	0.9503	0.9482–0.9525
F1-Score	0.9041	0.9019–0.9063
Specificity	0.7475	0.7414–0.7535
ROC-AUC	0.9457	0.9441–0.9474

PR-AUC 0.9661 0.9651–0.9672

This provided statistical uncertainty estimates instead of reporting only single-point metrics.

17. Threshold Stability Analysis

I evaluated thresholds around the selected value to determine whether the model was highly sensitive to small threshold changes.

The tested range was:

0.45 to 0.55

The robust region was:

0.45 to 0.50

Within this region:

- Recall remained at least 95%
- F1-Score remained at least 90%

Threshold 0.50 was selected because it was the upper edge of this robust region and provided better Precision and Specificity while still satisfying the Recall requirement.

18. Threshold Sensitivity Analysis

I also tested how much the main metrics changed when the threshold was changed by approximately ± 0.05 around 0.50.

The maximum observed changes were relatively small:

Metric	Maximum Absolute Change
Precision	0.0047
Recall	0.0091
F1-Score	0.0016
Specificity	0.0123

This indicated that the selected operating point was reasonably stable within the evaluated local threshold range.

19. Final Threshold Locking

After completing validation-based optimization and stability analysis, I locked the final threshold.

The final threshold was:

0.50

The selection rule was:

Select the highest threshold that satisfies the minimum Recall requirement of 95% while remaining within the robust threshold region identified during validation stability analysis.

The threshold was frozen before using the test set.

No test-set results were used to change this threshold.

20. Final Holdout Evaluation

After locking the threshold, I evaluated the model once on the previously untouched test set.

The test set contained:

55,741 samples

Final test results:

Metric	Test Result
Accuracy	86.00%
Precision	89.09%
Recall	88.41%
F1-Score	88.75%

Specificity 82.00%

ROC-AUC 94.51%

PR-AUC 96.69%

Final confusion matrix:

	Predicted Normal	Predicted Abnormal
Actual Normal	17,162	3,768
Actual Abnormal	4,036	30,775

Therefore:

- TN = 17,162
- FP = 3,768
- FN = 4,036
- TP = 30,775

The test set was used only for final holdout evaluation.

21. Validation vs Test Analysis

I compared validation and test performance after the threshold was locked.

Metric	Validation	Test
Accuracy	87.41%	86.00%
Precision	86.22%	89.09%
Recall	95.02%	88.41%
F1-Score	90.41%	88.75%
Specificity	74.74%	82.00%
ROC-AUC	94.57%	94.51%
PR-AUC	96.61%	96.69%

The most important observation was that ROC-AUC and PR-AUC remained very close between validation and test.

This indicates that the model's overall ranking/discrimination ability generalized well.

However, Recall decreased from 95.02% on validation to 88.41% on the unseen test set.

Therefore, the 95% Recall clinical target was not maintained on the final holdout set.

Because the threshold had already been locked before test evaluation, I did not retune the threshold using test data.

23. What I Personally Implemented

1. Loaded and analyzed the clinical dataset, including data quality, duplicates, class distribution, and leakage checks.
2. Removed unnecessary/leakage-prone features and prepared numerical and categorical variables.
3. Built the preprocessing pipeline using StandardScaler and OneHotEncoder.
4. Created stratified Train, Validation, and Test sets while keeping the test set untouched.
5. Trained Logistic Regression and evaluated the main classification metrics.
6. Implemented and compared Sigmoid and Isotonic probability calibration.
7. Swept thresholds from 0.00 to 1.00 and calculated Precision, Recall, F1, Specificity, FPR, and FNR.
8. Implemented Maximum-F1, Recall-constrained, Precision-constrained, and cost-sensitive threshold optimization.
9. Performed bootstrap confidence intervals, threshold stability, and sensitivity analysis.
10. Locked the final threshold at 0.50 using validation data.
11. Evaluated the locked threshold on the untouched test set and analyzed the results.
12. Documented the complete workflow and findings.

24. AI and ChatGPT Assistance

ChatGPT was used as a learning and coding assistant for:

1. Understanding threshold optimization, calibration, and clinical metrics.
2. Designing and reviewing the machine learning workflow.
3. Writing and troubleshooting Python and Scikit-learn code.
4. Checking preprocessing and data leakage.
5. Implementing threshold sweeping, clinical optimization, and cost analysis.
6. Implementing bootstrap validation and threshold stability analysis.

7. Interpreting validation and test results.
8. Reviewing the final methodology and documentation.

25. Prompts Used During Development

Project Development

Act as a senior AI/ML engineer and instructor. Explain each step, provide clean Google Colab code, wait for my actual output, and do not make up results or skip data leakage analysis.

Threshold Optimization

Systematically vary the classification threshold from 0.00 to 1.00, evaluate Precision, Recall, and F1-Score, visualize the trade-offs, and identify the threshold that matches the clinical priority.

Technical Review

Check the implementation for technical problems, data leakage, incorrect calculations, and inconsistencies before continuing.

Final Review

Review the complete project for preprocessing, calibration, threshold optimization, statistical validation, threshold locking, final evaluation, results, and limitations.

26. How AI Was Used

AI was used as a supporting tool, not as a replacement for my implementation.

I personally executed the code in Google Colab, checked the actual outputs, evaluated the results, and made the final threshold decision.

ChatGPT provided concept explanations, coding guidance, troubleshooting, technical review, result interpretation, and documentation support.